

Defects4J Lang: Survival of LLM-Generated Tests vs Developer Tests

This week the survival experiment moved from zod to the Defects4J dataset of the prompting-techniques paper (Ouédraogo et al., EMSE 2026). Its generation pipeline was rebuilt with a different model, and the number of LLM tests was raised to the number of developer tests before measuring survival. Date: 2026-10-03.

1. Questions

- When the LLM test count is raised to the developer test count, how many tests of each kind still pass, unchanged, at later versions of the code?
- At the version that fixes the bug, which LLM tests fail, and why?

2. Setup

- Subject:** the 14 Lang records of the paper's Defects4J dataset (9 classes). t = the buggy version of each record. Later points: the record's own fixed version, then every dataset Lang bug version dated after t (1 to 14 points per record; 105 record/point pairs, 60 with the class present. The five Lang 2.x records lose their class at the first Lang 3.x point, where the package moved from lang to lang3).
- Generation:** the paper's five prompts (ZSL, FSL, CoT, ToT, GToT), each given the class source as in the paper; gpt-oss-120b via OpenRouter, temperature 0.7 (the paper's four models are no longer served). The paper's pipeline (extract, compile, run, JaCoCo) is followed; only its extraction step was replaced, because the released reimplementation rejected fenced code and mistook generics for HTML. Generation was repeated per record until the LLM methods passing at t reached the developer methods passing at t; methods with an identical body were counted once.
- Developer tests:** the Defects4J tests .relevant classes of each bug (49 classes), source frozen at t. Unit: the test method. Survival = passes unchanged at a later point. Methods that fail at t (the Defects4J trigger tests) are excluded from both baselines.

3. Results

Table 1. Populations at t and the outcome at the fixed version.

Record / class	Developer methods	LLM methods (identical bodies removed)	Generations per prompt	LLM failing at fixed
Lang-4 LookupTranslator	136	141 (2)	3	6
Lang-5 LocaleUtils	12	100 (30)	1	0
Lang-6 CharSequenceTranslator	141	180 (1)	4	3
Lang-11 RandomStringUtils	10	92 (12)	1	0
Lang-12 RandomStringUtils	8	81 (6)	1	0
Lang-13 SerializationUtils	207	230 (53)	6	0
Lang-17 CharSequenceTranslator	44	49 (0)	1	1
Lang-19 NumericEntityUnescaper	30	53 (0)	1	10
Lang-28 NumericEntityUnescaper	19	61 (0)	1	1
Lang-43 ExtendedMessageFormat	6	38 (0)	1	0
Lang-54 LocaleUtils	11	62 (14)	1	0
Lang-55 Stopwatch	5	65 (1)	1	1
Lang-57 LocaleUtils	0*	104 (35)	1	0
Lang-64 ValuedEnum	74	97 (2)	3	0
All	703	1,353 (156)		22

* Lang-57: all 11 developer methods fail at t (the bug is in static initialisation reached from setUp), so the record has no developer baseline. Developer methods failing at the fixed version: 0 of 703.

Table 2. Survival by days after t (method × time point; absent points excluded from the denominator).

Days after t	Dev pass / present	Dev compile-fail	Dev assertion fail	LLM pass / present	LLM compile-fail	LLM assertion fail, error
0 (fixed version)	703 / 703 (100.0%)	0	0	1,331 / 1,353 (98.4%)	0	22
1 to 30	38 / 38 (100.0%)	0	0	125 / 134 (93.3%)	0	9
31 to 180	838 / 879 (95.3%)	41	0	1,553 / 1,560 (99.6%)	0	7
181 to 365	639 / 746 (85.7%)	97	6	1,179 / 1,209 (97.5%)	0	30
366 to 730	504 / 540 (93.3%)	28	6	1,009 / 1,072 (94.1%)	0	63
731 and later	90 / 131 (68.7%)	41	0	256 / 280 (91.4%)	0	24

Developer rows at 181 to 730 days also contain 6 methods not run (inherited methods missing from the JUnit listing). Points beyond 365 days exist for few records.

4. Observations

- At the fixed version, developer tests do not fail and 22 LLM tests do; all 22 execute lines changed by the fix.** By their failure messages, 19 of the 22 assert the behaviour the fix removed: nine Lang-19 tests expect the `StringIndexOutOfBoundsException` that the fix removes, six Lang-4 tests assert that a non-String key does not match, and four Lang-6 and Lang-28 tests assert the old handling of surrogate pairs. Of the other three, one is a timing test (`Thread.sleep`) that passes when rerun, one errors with an index exception in the fixed code, and one asserts a consumed-character count. These methods keep failing at every later point; the only LLM failures that appear later are 8 further Lang-28 methods, in the same unescaper class, about a year after t.
- The two populations fail in different ways, which follows from their scope.** Developer failures at later versions are compile failures: 207 method/point rows, all "cannot find symbol", from three test classes of other classes whose API changed (two exception test classes of Lang-13, the enum test class of Lang-64), plus one assertion method (`TypeUtilsTest.testIsAssignable`). LLM tests never fail to compile; every LLM failure is an assertion failure or an exception. The LLM tests reference only the class the prompt passed; the developer relevant tests reference the project.
- Raising the LLM count to the developer count took little:** after one generation per prompt, 10 of 14 records already exceeded the developer count, and the other four reached it after 2 to 5 further generations. 156 of 1,509 passing LLM methods (10.3%) had a body identical to an earlier one and were removed.

5. Limitations

- The model differs from the paper's; only the paper's pipeline is reproduced, not its numbers. Variation between generation runs was not measured.
- 14 records of one project; 45 of 105 record/point pairs are unmeasurable after the package move. The public API of the nine classes did not change within the measured points, which contributes to the LLM compile-failure count of zero.
- "Executes a changed line" is a coverage rule, not proof that the fix caused the failure; these classes are small, so most tests execute the changed lines. Duplicate removal matched identical bodies only; whether differently worded tests check different behaviour was not measured.

Repository: Hamjoon/llm4ts-replication, branch experiment/2026-10-week1-d4j-replication (results/p2-survival-methods.csv, results/p2-survival-summary.md, docs/).